AVEVA Partner Ecosystem | System Integrator

ENDORSED

DOCUMENTACIÓN DE CÓDIGO

COINTAC

PPSW

PI Point Status Watchtower

Índice

1 Estructura del Proyecto 3
2 Catálogo de Clases 4
2.1 ConfigManager — Gestión de Configuración 4
2.2 PiPointStatusWatchtowerOperation — Bucle Principal 4
2.3 MonitoringProcessOrchestrator — Orquestador Central 4
2.4 TypeValueCounterService — Conteo de Estados 5
2.5 RecoverPIPointsService — Recuperación de PI Points 5
2.6 Patrones de Diseño Aplicados 5
3 Modelos de Datos 6
3.1 Config — Raíz de Configuración 6
3.2 MonitoringProfile — Perfil de Calidad 6
3.3 Modelos de Resultado 6
4 Servicios de Calidad (Quality Checks) 7
4.1 StaleCheckService — Frescura de Datos 7
4.2 FlatlineCheckService — Detección de Congelamiento 7
4.3 DensityCheckService — Densidad de Eventos 8
5 Adaptadores de Escritura (Metrics Adapters) 8
6 Buenas Prácticas Implementadas 8
7 Guía de Mantenimiento del Código 9
7.1 Agregar un nuevo módulo de calidad 9
7.2 Agregar un nuevo modo de agrupación 9
8 Control de versiones 9

© 2026 Contac Ingenieros LTDA. Todos los derechos reservados. Documento confidencial. Prohibida su reproducción o distribución sin autorización previa. | Jorge Alessandri 210, La Reina, Santiago, Chile | https://www.contac.cl/

1 Estructura del Proyecto

```
— Program.cs                    # Punto de entrada
— ProjectInstaller.cs           # Instalador del servicio Windows
— packages.config               # Dependencias NuGet
— NLog.config                   # Configuración de logging (base)
— App.config                    # Configuración .NET Framework
— PiPointStatusWatchtower.csproj  # Archivo de proyecto
— src/
—— resources/
——— appsettings.yml             # Configuración principal del servicio
—— main/
——— WinServicePiPointStatus.cs  # Clase del servicio Windows
——— core/
———— operations/
————— PiPointStatusWatchtowerOperation.cs  # Bucle principal
——— application/
———— orchestrators/
————— MonitoringProcessOrchestrator.cs  # Orquestador central
———— services/
————— legacy/
————— process/
————— ProcessLegacy.cs  # Controlador Legacy
————— features/
————— pointSource/
————— check/
————— CheckPointSource.cs # Verificador PointSource
————— operation/
————— OperationPointSource.cs # Operación PointSource
————— interfaceID/
————— check/
————— CheckInterfaceID.cs  # (Reservado)
————— operation/
————— OperationInterfaceID.cs # (Reservado)
————— business/
————— process/
————— ProcessBusiness.cs  # Controlador Business
————— common/
————— monitoring/
————— ResolveMonitoringProfile.cs # Resolución de perfiles
————— features/
————— standard/
————— operation/
————— OperationStandard.cs # Descubrimiento dinámico
————— staticExDesc/
————— operation/
————— OperationStatic.cs  # Contextos estáticos
——— services/
———— counter/
————— TypeValueCounterService.cs  # Conteo de estados
———— quality/
————— StaleCheckService.cs  # Frescura (RF-07)
————— FlatlineCheckService.cs  # Congelamiento (RF-08)
————— DensityCheckService.cs  # Densidad (RF-09)
———— parser/
————— ExDescParserService.cs  # Parseo de ExDesc
———— recover/
————— RecoverPIPointsService.cs  # Recuperación de PI Points
———— write/
————— WriteValueToPITagService.cs # Escritura a PI Tags
———— af/
————— AFStructureService.cs  # Validación AF (comentado)
——— infrastructure/
———— connections/
————— PIServerConnectionManager.cs # Pool de conexiones PI
———— adapters/
————— PIMetricsAdapter.cs  # Adaptador métricas base
————— PIStaleMetricsAdapter.cs  # Adaptador métricas Stale
————— PIFlatlineMetricsAdapter.cs # Adaptador métricas Flatline
————— PIDensityMetricsAdapter.cs  # Adaptador métricas Density
——— models/
———— AppSettingsModel.cs  # Modelos de configuración
———— GroupingMode.cs  # Enum de modos de agrupación
———— TypesValueAndStateCountsModel.cs # Modelo conteo de estados
———— StaleCheckResult.cs  # Modelo resultado Stale
———— FlatlineCheckResult.cs  # Modelo resultado Flatline
———— DensityCheckResult.cs  # Modelo resultado Density
——— config/
———— ConfigManager.cs  # Singleton de configuración
———— LoggerConfig.cs  # Configuración de NLog
——— constants/
———— Constants.cs  # Constantes globales
———— MessageCode.cs  # Catálogo de mensajes
——— utils/
———— StringUtils.cs  # Utilidades de formato
———— LoggerUtils.cs  # Logging condicional
```

© 2026 Contac Ingenieros LTDA. Todos los derechos reservados. Documento confidencial. Prohibida su reproducción o distribución sin autorización previa. | Jorge Alessandri 210, La Reina, Santiago, Chile | https://www.contac.cl/

2 Catálogo de Clases

2.1 ConfigManager — Gestión de Configuración

Patrón: Singleton (**Lazy<T>** thread-safe).

Responsabilidades:

- Carga y deserializa **appsettings.yml** al iniciar usando **YamlDotNet**.
- Valida campos obligatorios (**PIServerName**, **ExecutionIntervalInMinutes**, **PointSources**, **EnabledStates**).
- Si la validación falla, registra el error y detiene el proceso con **Environment.Exit(1)**.
- Provee acceso global a **CurrentConfig** y **Logger**.

Propiedades públicas:

Propiedad	Tipo	Descripción
Instance	ConfigManager	Instancia singleton.
CurrentConfig	Config	Configuración deserializada desde YAML.
Logger	NLog.Logger	Instancia del logger para toda la aplicación.

2.2 PiPointStatusWatchtowerOperation — Bucle Principal

Responsabilidades:

- Gestiona el ciclo de vida del bucle de ejecución con **CancellationTokenSource**.
- Calcula puntos de ejecución alineados al intervalo configurado.
- Delega cada ciclo a **MonitoringProcessOrchestrator.StartAsync()**.
- Captura excepciones por ciclo sin detener el servicio.

Métodos públicos:

Método	**Descripción**
StartLoopPointSource()	Inicia el bucle asíncrono de monitoreo.
StopLoopPointSource()	Cancela el bucle y desconecta los servidores PI.

2.3 MonitoringProcessOrchestrator — Orquestador Central

Responsabilidades:

- Establece conexiones a PI Data Archive (lectura y escritura).
- Instancia **WriteValueToPITagService** para escritura de métricas.

© 2026 Contac Ingenieros LTDA. Todos los derechos reservados. Documento confidencial. Prohibida su reproducción o distribución sin autorización previa. | Jorge Alessandri 210, La Reina, Santiago, Chile | https://www.contac.cl/

- Redondea el timestamp al minuto más cercano.
- Invoca secuencialmente **ProcessLegacy** y luego **ProcessBusiness**.

2.4 TypeValueCounterService — Conteo de Estados

Tipo: Servicio estático.

Método principal: TypeValueCounter(piPoints, counterDuplicate, pointSource, isFirstExecution)

Comportamiento:

1. Procesa PI Points en lotes de 15,000 usando **PIPointList.CurrentValue()**.
2. Para cada valor Bad, busca el código de estado en el diccionario invertido **ReverseStates** (valor numérico → nombre del estado).
3. Si **counterDuplicate = true**, ejecuta **CounterDuplicate()** que agrupa por **InstrumentTag** (case-insensitive) y retorna el total de duplicados.

2.5 RecoverPIPointsService — Recuperación de PI Points

Métodos:

Método	Filtro Utilizado	Uso
RecoverPIPoints(pointSource, piServer)	PointSource = valor	Modo Legacy.
RecoverPIPointsByInterfaceID(interfaceId, piServer)	Location1 = valor	Reservado.
RecoverPIPointsByAttribute(attrName, attrValue, piServer)	atributo = valor exacto	Modo Static.
RecoverPIPointsByContainsValue(attrName, attrValue, piServer)	atributo = *valor* (comodín)	Modo Standard.

2.6 Patrones de Diseño Aplicados

Patrón	Implementación	Beneficio
Singleton	ConfigManager con Lazy<T>	Acceso global thread-safe a la configuración sin cargas repetidas.
Strategy	ProcessBusiness selecciona OperationStandard u OperationStatic según DiscoveryMode.	Extensibilidad para nuevos modos de descubrimiento.
Template Method	OperationStandard y OperationStatic comparten la secuencia: recuperar → contar → resolver perfil → ejecutar calidad → escribir.	Consistencia en el flujo de procesamiento.
Adapter	PIMetricsAdapter, PIStaleMetricsAdapter, PIFlatlineMetricsAdapter, PIDensityMetricsAdapter.	Desacopla la lógica de cálculo de la lógica de escritura/nomenclatura.

© 2026 Contac Ingenieros LTDA. Todos los derechos reservados. Documento confidencial. Prohibida su reproducción o distribución sin autorización previa. | Jorge Alessandri 210, La Reina, Santiago, Chile | https://www.contac.cl/

Connection Pool (simplificado)	PIServerConnectionManager con Dictionary<string, PIServer>.	Reutilización de conexiones y prevención de aperturas múltiples.
Feature Flags	MonitoringProfile con propiedades booleanas por módulo.	Activación granular de funcionalidades sin cambios de código.

3 Modelos de Datos

3.1 Config — Raíz de Configuración

```
public class Config
{
  public AppConfig AppConfig { get; set; }
  public LegacyMonitoringConfig LegacyMonitoring { get; set; }
  public BusinessMonitoringConfig BusinessMonitoring { get; set; }
  public Dictionary<string, MonitoringProfile> MonitoringProfiles { get; set; }
  public Dictionary<string, int> EnabledStates { get; set; }
  public Dictionary<string, int> DisabledStates { get; set; }
}
```

3.2 MonitoringProfile — Perfil de Calidad

```
public class MonitoringProfile
{
  public bool EnableStaleCheck { get; set; }      // Frescura
  public bool EnableFlatlineCheck { get; set; }   // Congelamiento
  public bool EnableDensityCheck { get; set; }    // Densidad
  public List<int> LatencyBuckets { get; set; }   // Umbrales en minutos
  public int FlatlineSampleCount { get; set; }    // Eventos a analizar (min: 2)
  public int MinEventsPerHour { get; set; }       // Umbral baja densidad (0 = off)
  public int MaxEventsPerHour { get; set; }       // Umbral alta densidad (0 = off)
}
```

3.3 Modelos de Resultado

Modelo	Propiedades Clave	Servicio Productor
TypesValueAndStateCountsModel	BadPointsCount, DuplicateCount, StateCounts (diccionario estado→conteo)	TypeValueCounterService
StaleCheckResult	BucketCounts (diccionario umbral→conteo), MaxGroupLatencyMinutes, TotalTagsEvaluated, TotalTagsExcluded	StaleCheckService
FlatlineCheckResult	FrozenCount, TotalTagsEvaluated, TotalTagsExcluded	FlatlineCheckService

© 2026 Contac Ingenieros LTDA. Todos los derechos reservados. Documento confidencial. Prohibida su reproducción o distribución sin autorización previa. | Jorge Alessandri 210, La Reina, Santiago, Chile | https://www.contac.cl/

DensityCheckResult	LowDensityCount, HighDensityCount, TotalTagsEvaluated, TotalTagsExcluded	DensityCheckService

4 Servicios de Calidad (Quality Checks)

4.1 StaleCheckService — Frescura de Datos

Algoritmo detallado:

PARA CADA lote de 15,000 PI Points:

1. Crear PIPointList y obtener CurrentValue() en batch.

2. PARA CADA AFValue del resultado:

a. Si !IsGood → TotalTagsExcluded++ → CONTINUAR

b. TotalTagsEvaluated++

c. lagMinutes = (UtcNow - Snapshot.Timestamp).TotalMinutes

d. Si lagMinutes > maxLagMinutes → maxLagMinutes = lagMinutes

e. PARA CADA bucket en sortedBuckets:

Si lagMinutes > bucket → BucketCounts[bucket]++

4. MaxGroupLatencyMinutes = maxLagMinutes

Nota técnica: Los contadores de bucket son acumulativos. Un tag con Lag = 100 min y buckets [15, 60, 240] incrementará Level1 y Level2, pero no Level3.

4.2 FlatlineCheckService — Detección de Congelamiento

Reglas de exclusión:

Regla	Condición	Razón
Regla 1	PointType ∈ {Digital, String, Blob}	Estos tipos no representan valores numéricos continuos.
Regla 2	Step = 1	Señal escalonada legítima (no es un "congelamiento").
Regla 3	ExDesc contiene ;IGNORE_FLATLINE;	Exclusión explícita por el administrador.

Algoritmo de Welford (estabilidad numérica):

mean = 0, m2 = 0, n = 0

© 2026 Contac Ingenieros LTDA. Todos los derechos reservados. Documento confidencial. Prohibida su reproducción o distribución sin autorización previa. | Jorge Alessandri 210, La Reina, Santiago, Chile | https://www.contac.cl/

PARA CADA valor en valores:

n++

delta = valor - mean

mean += delta / n

delta2 = valor - mean

m2 += delta × delta2

varianza = m2 / n

$\sigma = \sqrt{varianza}$

SI $\sigma < 1 \times 10^{-5}$ *→ TAG ES FROZEN*

4.3 DensityCheckService — Densidad de Eventos

Regla de exclusión: ExDesc contiene **;IGNORE_DENSITY;**.

Ventana temporal: Última hora (**UtcNow - 1h** a **UtcNow**).

Clasificación:

- **eventCount < MinEventsPerHour** → **Baja densidad** (posible falta de comunicación o muestreo insuficiente).
- **eventCount > MaxEventsPerHour** → **Alta densidad** (posible ruido, scan rate excesivo).

5 Adaptadores de Escritura (Metrics Adapters)

Todos los adaptadores comparten el mismo patrón de resolución del nombre del servidor de escritura:

var *piServerName = ConfigManager.Instance.CurrentConfig.AppConfig.PIServerWriteName*

?? ConfigManager.Instance.CurrentConfig.AppConfig.PIServerName;

Si **PIServerWriteName** no está configurado, se usa **PIServerName** como fallback.

La escritura se realiza a través de **WriteValueToPITagService**, que opera como sigue:

1. Busca el PI Tag por nombre completo: **PIPoint.FindPIPoint()**.
2. Si el tag existe → escribe el valor con **AFUpdateOption.Insert** y **AFBufferOption.Buffer**.
3. Si el tag **no** existe → registra un **Warning** en el log (no falla).

6 Buenas Prácticas Implementadas

Práctica	Implementación
Procesamiento por lotes	Todas las operaciones masivas (CurrentValue, LoadAttributes, RecordedValues) usan lotes de 15,000 elementos para prevenir timeouts de PI.
Logging condicional	Los logs detallados (ServiceDetails = true) solo se generan cuando están explícitamente activados, reduciendo I/O en producción.

© 2026 Contac Ingenieros LTDA. Todos los derechos reservados. Documento confidencial. Prohibida su reproducción o distribución sin autorización previa. | Jorge Alessandri 210, La Reina, Santiago, Chile | https://www.contac.cl/

Tolerancia a fallos	Cada tag, cada grupo y cada ciclo tiene su propio bloque try/catch. Un error en un elemento no impide el procesamiento de los siguientes.
Separación de lectura/escritura	Servidores PI independientes para lectura y escritura, evitando contención de recursos en el Data Archive de producción.
Inmutabilidad de configuración	La configuración se carga una sola vez al inicio y permanece inmutable durante la ejecución del servicio.
Documentación XML	Las clases y métodos principales incluyen documentación XML (<summary>, <param>, <returns>) para generación automática de documentación.

7 Guía de Mantenimiento del Código

7.1 Agregar un nuevo módulo de calidad

Para agregar un nuevo análisis de calidad (ej. "Jitter Check"):

a) Crear el modelo de resultado en models/ (ej. JitterCheckResult.cs).
b) Crear el servicio de evaluación en services/quality/ (ej. JitterCheckService.cs) con método Evaluate().
c) Crear el adaptador de escritura en infrastructure/adapters/ (ej. PIJitterMetricsAdapter.cs).
d) Agregar el Feature Flag en MonitoringProfile (ej. EnableJitterCheck).
e) Agregar la invocación condicional en OperationStandard.cs y OperationStatic.cs.
f) Agregar los parámetros en appsettings.yml bajo MonitoringProfiles.

7.2 Agregar un nuevo modo de agrupación

a) Crear un nuevo paquete en application/services/legacy/features/ o business/features/.
b) Implementar las clases Check* y Operation* siguiendo el patrón existente.
c) Agregar el caso al switch en ProcessLegacy.cs o ProcessBusiness.cs

8 Control de versiones

Versión	Responsable	Fecha de entrega
1	Ing. Ivan Antony Sánchez Chávez	20-04-2026

© 2026 Contac Ingenieros LTDA. Todos los derechos reservados. Documento confidencial. Prohibida su reproducción o distribución sin autorización previa. | Jorge Alessandri 210, La Reina, Santiago, Chile | https://www.contac.cl/